HDOC Day-45

Aim

To write a C program that:

- Reads a variable-length integer array.
- Sorts it in **ascending order using Bubble Sort**.
- Passes the array using **pointers**.
- Finds the **first and last occurrence** of a target element.
- Prints `-1, -1` if the target is absent.

C Program

```
#include <stdio.h>

void Bubble_sort(int *arr, int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }
}

void occurrence(int *arr, int n)
{
    int target, i;
    int first = -1, last = -1;

    printf("Enter target: ");
    scanf("%d", &target);

    for (i = 0; i < n; i++)
    {
        if (*(arr + i) == target)
        {
            if (first == -1)
                first = i;

            last = i;
        }
    }
```

```
    if (first == -1)
    {
        printf("-1, -1\n");
    }
    else
    {
        printf("First occurrence: %d at index %d\n",
               *(arr + first), first);

        printf("Last occurrence: %d at index %d\n",
               *(arr + last), last);
    }
}

int main()
{
    int n, i;

    printf("Enter size of array: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter %d elements:\n", n);

    for (i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    Bubble_sort(nums, n);

    printf("Sorted array: ");

    for (i = 0; i < n; i++)
    {
        printf("%d ", nums[i]);
    }

    printf("\n");

    occurrence(nums, n);

    return 0;
}
```

Sample Outputs-

```
┌──(kali㉿kali)-[~/Desktop]
└─$ cc day46.c

┌──(kali㉿kali)-[~/Desktop]
└─$ ./a.out
Enter size of array: 6
Enter 6 elements:
5 7 7 8 8 10

Sorted array: 5 7 7 8 8 10
Enter target: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 4

┌──(kali㉿kali)-[~/Desktop]
└─$ ./a.out
Enter size of array: 5
Enter 5 elements:
5 7 7 8 10
Sorted array: 5 7 7 8 10
Enter target: 8
First occurrence: 8 at index 3
Last occurrence: 8 at index 3
```

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./a.out
Enter size of array: 6
Enter 6 elements:
5 7 7 8 8 10
Sorted array: 5 7 7 8 8 10
Enter target: 6
-1, -1
```

Result

Thus, the program successfully sorts a variable-length integer array using the **Bubble Sort technique** and uses **pointers** to pass the array to `Bubble_sort()` and `occurrence()`. It correctly finds the **first and last occurrence** of the target element and displays `-1, -1` when the target is absent.